NEXIPLAY CORE API AES-256-CBC ENCRYPTED ZERO ORIGIN LEAK

NexiPlay Protected Anime Stream API

Official Developer Integration Guide, Architecture & Universal Player Specification

🔒 **Security & Zero-Leak Architecture:**

This API allows third-party developers, mobile apps, and external websites to stream 100% of all anime catalog titles via a single unified endpoint. The upstream origin hosts (e.g. `codedew.com`, `razorshell.space`, `*.groovy.monster`) are **never** leaked to client network inspectors. All requests, playlist manifests (`.m3u8`), and video chunks (`.ts`) are proxied and tunneled through your NexiPlay instance with dynamic 2-hour encrypted security tokens.

1. System Architecture Overview

The streaming pipeline operates across two lightweight, high-performance layers:

```
[Your Client / Player / Mobile App]
    |
    ▼ 1. GET /api/v1/anime/stream?slug={anime}&season=1&episode=1
[NexiPlay Protected Stream Gateway]
    ├── Checks Anime & Episode in Database
    ├── Server-side JavaScript Virtual Machine deobfuscates JuicyCodes & extracts master HLS
    ├── Encrypts target stream with AES-256-CBC & 2-Hour TTL token
    └── Returns JSON containing playbackUrl and security token
    |
    ▼ 2. Video Player loads: GET /api/v1/anime/playback?token={token}
[NexiPlay Tunneled Playback Proxy]
    ├── Decrypts token & verifies signature & expiry timestamp
    ├── Rewrites master & variant .m3u8 playlists pointing to proxy routes
    └── Serves .ts video segments with 206 Partial Content & Full CORS headers
    |
    ▼
[Smooth, Ad-Free, 1080p / 720p HLS Video Plays Without Any Iframe!]
```

2. The Unified Stream API Endpoint

HTTP Method: `GET`

Endpoint URL: `https://your-domain.com/api/v1/anime/stream`

CORS Policy: `Access-Control-Allow-Origin: *` (Allows browser cross-origin calls)

Query Parameters

Parameter	Type	Default	Description
`slug`	String	Required*	Anime URL slug (e.g. `blue-box-2026`, `hana-kimi-2026`).
`id`	UUID	Optional*	Database ID of anime (either `slug` or `id` must be provided).
`season`	Integer	`1`	Season number (1, 2, 3...).
`episode`	Integer	`1`	Episode number (1, 2, 3... or aliases: `ep`).

3. API Response Specification

Calling the endpoint returns a JSON payload with metadata, expiration, token, and the ready-to-play URL:


```
{
  "success": true,
  "title": "Blue Box",
  "type": "anime",
  "slug": "blue-box-2026",
  "season": 1,
  "episode": 1,
  "server": "RR Nexiplay Server",
  "format": "hls_stream",
  "expiresInSeconds": 7200,
  "token": "b5870d09f28e80c144a1976b581c760c:0b9b2b4ed354a9cf4115455363...",
  "playbackUrl": "https://your-domain.com/api/v1/anime/playback?token=b5870d09f28e...",
  "embedUrl": "https://your-domain.com/embed/rr?slug=blue-box-2026&season=1&episode=1",
  "iframeCode": "<iframe src=\"https://your-domain.com/embed/rr?...\" width=\"100%\"...></iframe>"
}
```


Key Field Definitions

Field	Type	Purpose
`playbackUrl`	URL String	**Primary Stream Target:** Pass this directly into Video.js, Plyr, Hls.js, ExoPlayer, or AVPlayer. No iframe needed!
`token`	Hex String	AES-256-CBC token encoding the target stream and session state. Valid for 2 hours.
`format`	String	`hls_stream` (Master HLS playlist for native video playback).
`embedUrl`	URL String	Optional responsive full-screen web embed player URL.

4. Client Integration 1: Vanilla HTML5 & HLS.js (No Framework)

To stream directly in any web browser without an iframe:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Custom Anime Player</title>
  <script src="https://cdn.jsdelivr.net/npm/hls.js@latest"></script>
  <style>
    .player-container { width: 100%; max-width: 960px; margin: 40px auto; aspect-ratio: 16/9; background: #00
0; border-radius: 12px; overflow: hidden; }
    video { width: 100%; height: 100%; object-fit: contain; }
  </style>
</head>
<body>
  <div class="player-container">
    <video id="videoPlayer" controls playsinline></video>
  </div>

  <script>
    async function playAnime(slug, season = 1, episode = 1) {
      const video = document.getElementById('videoPlayer');
      const apiUrl = `https://your-domain.com/api/v1/anime/stream?slug=${slug}&season=${season}&episode=${epi
sode}`;

      try {
        const res = await fetch(apiUrl);
        const data = await res.json();
        if (!data.success || !data.playbackUrl) {
          alert('Stream error: ' + (data.error || 'Failed to resolve stream'));
          return;
        }

        const streamUrl = data.playbackUrl;

        // Chrome, Firefox, Edge, Android Browsers:
        if (Hls.isSupported()) {
          const hls = new Hls({ enableWorker: true });
          hls.loadSource(streamUrl);
          hls.attachMedia(video);
          hls.on(Hls.Events.MANIFEST_PARSED, () => video.play());
        }
        // Safari / iOS Native HLS:
        else if (video.canPlayType('application/vnd.apple.mpegurl')) {
          video.src = streamUrl;
          video.addEventListener('loadedmetadata', () => video.play());
        }
      } catch (err) {
        console.error('Playback fetch failed:', err);
      }
    }

    // Launch target anime:
    playAnime('blue-box-2026', 1, 1);
  </script>
</body>
</html>
```

5. Client Integration 2: React / Next.js Component

Drop-in TypeScript component for modern React, Next.js, or Remix applications:

```tsx
// components/AnimeStreamPlayer.tsx
'use client';

import { useEffect, useRef, useState } from 'react';
import Hls from 'hls.js';

interface PlayerProps {
  slug: string;
  season?: number;
  episode?: number;
  apiBaseUrl?: string; // e.g. "https://your-domain.com"
}

export default function AnimeStreamPlayer({ slug, season = 1, episode = 1, apiBaseUrl = '' }: PlayerProps) {
  const videoRef = useRef<HTMLVideoElement>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let hls: Hls | null = null;
    setLoading(true);
    setError(null);

    async function initStream() {
      try {
        const res = await fetch(
          `${apiBaseUrl}/api/v1/anime/stream?slug=${encodeURIComponent(slug)}&season=${season}&episode=${epis
ode}`
        );
        const data = await res.json();
        if (!data.success || !data.playbackUrl) {
          throw new Error(data.error || 'Stream not available');
        }

        const video = videoRef.current;
        if (!video) return;

        if (Hls.isSupported()) {
          hls = new Hls({ enableWorker: true });
          hls.loadSource(data.playbackUrl);
          hls.attachMedia(video);
          hls.on(Hls.Events.MANIFEST_PARSED, () => {
            setLoading(false);
            video.play().catch(() => {});
          });
        } else if (video.canPlayType('application/vnd.apple.mpegurl')) {
          video.src = data.playbackUrl;
          video.onloadedmetadata = () => {
            setLoading(false);
            video.play().catch(() => {});
          };
        }
      } catch (err: any) {
        setError(err.message || 'Playback error');
        setLoading(false);
      }
    }

    initStream();
    return () => { if (hls) hls.destroy(); };
  }, [slug, season, episode, apiBaseUrl]);
```

```
  return (
    <div className="relative w-full aspect-video bg-black rounded-2xl overflow-hidden shadow-2xl">
      {loading && (
        <div className="absolute inset-0 flex items-center justify-center bg-black/60 text-white z-10">
          <div className="w-8 h-8 border-2 border-red-500 border-t-transparent rounded-full animate-spin" />
        </div>
      )}
      {error && (
        <div className="absolute inset-0 flex items-center justify-center bg-black text-red-400 p-4 z-10 font
-bold">
          {error}
        </div>
      )}
      <video ref={videoRef} controls playsInline className="w-full h-full object-contain" />
    </div>
  );
}
```

6. Client Integration 3: Mobile Apps (Flutter / Android)

> **Mobile App Advantage:** Mobile apps should never use web iframes because webviews leak memory and have choppy playback. With `playbackUrl`, Flutter, React Native, and Android apps load streams directly via native media hardware decoders (ExoPlayer / AVPlayer).

Flutter (using `video_player` package):

```
// 1. Fetch metadata
final res = await http.get(Uri.parse(
  'https://your-domain.com/api/v1/anime/stream?slug=blue-box-2026&season=1&episode=1'
));
final data = jsonDecode(res.body);
final String playbackUrl = data['playbackUrl'];

// 2. Feed stream into native player
_controller = VideoPlayerController.networkUrl(
  Uri.parse(playbackUrl),
  formatHint: VideoFormat.hls,
)..initialize().then((_) {
  setState(() {});
  _controller.play();
});
```

7. Zero-Code HTML Iframe Embed (For WordPress / Blogger)

For bloggers, non-coders, or simple iframe embeds, use the auto-generated `embedUrl`:

```
<iframe
  src="https://your-domain.com/embed/rr?slug=blue-box-2026&season=1&episode=1"
  width="100%"
  height="500px"
  frameborder="0"
  allowfullscreen
  allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; fullscree
n">
</iframe>
```

8. Error Handling & HTTP Status Codes

Status Code	Error Reason	Resolution
`200 OK`	Success	Stream token and `playbackUrl` generated successfully.
`400 Bad Request`	Missing parameters	Provide at least `slug` or `id` in query string.
`403 Forbidden`	Invalid or expired token	Token has passed its 2-hour TTL. Re-query `/api/v1/anime/stream` for a fresh token.
`404 Not Found`	Anime or episode not found	Verify anime slug or season/episode number exists in the database.
`500 Server Error`	Extractor timeout / error	The upstream deobfuscator encountered a network issue; client should retry with exponential backoff.

9. Token Security & Best Practices

- **Token Rotation:** Every token is strictly tied to an expiration timestamp (120 minutes). If a user pauses a video and returns 3 hours later, your client application should catch `403` and re-fetch a new stream URL.
- **Hotlink & Source Protection:** Because all video segment URLs are rewritten to proxy through `/api/v1/anime/playback`, clients can never extract the origin host. Sharing raw segment links without a valid active token results in an immediate `403 Forbidden`.
- **Cross-Origin Resource Sharing (CORS):** Both the stream metadata API and the playback chunk proxy provide `Access-Control-Allow-Origin: *`, allowing integration into websites hosted anywhere (Netlify, Vercel, WordPress, GitHub Pages, or local apps).